Annahi Bermudez

CMPSC 448

Midterm Project 1

Repo Link: https://github.com/Annahi05/llm_identifier_448

WHO WROTE IT? IDENTIFYING LLMS FROM THEIR RESPONSES

I. PROBLEM AND DATASET

Problem

As AI becomes more commonly used in everyday life, it is increasingly important to understand whether we can identify which language model generated a given piece of text. Different LLMs may have their own writing styles, vocabulary, and formatting habits. This project tests this idea by asking whether machine learning can pick up on these differences and use them to predict the source of a response.

To do this, I used two neural network models, a CNN and an LSTM, and compared them against baseline models. Throughout the process, this report answers these four questions: (RQ1) Can an AI family be identified from its response? (RQ2) Can a user's question help identify the AI? (RQ3) For questions from different domains, do digital fingerprints carry over? (RQ4) What distinguishing features of the text differentiate the AIs?

Data Source

For this project, I used the PersonalLLM dataset from Hugging Face, released under the Creative Commons Attribution 4.0 license. This set contained questions and responses from 8 different AI models in six families: Anthropic (Claude 3 Opus, Claude 3 Sonnet), OpenAI (GPT-4 Turbo, GPT-4o), Google (Gemini Pro 1.5), Meta (Llama 3 70B Instruct), Mistral (Mixtral 8x22B Instruct), and Cohere (Command R+). I converted the data into one row per response, containing the AI's name, the question, and the response, following the required (LLM_name, LLM_input, LLM_output) order. Since the main task was to predict the 6 families, both Claude models are labeled Anthropic.

Cleaning and Balancing

To prepare the data, I combined the original training and test splits into a total of 10,402 questions. I then filtered out any responses shorter than five words and removed any duplicates that may be present. Afterwards, I reduced the number of responses so that each of the eight models would have the same 10,283 number of responses, for a total of

82,264. It must be noted that since Anthropic and OpenAI each contain two models, those families will have twice as many responses as the other four, and thus, a classifier that always predicts the larger family receives a 24.9%.

Splits

When splitting the data, an 80/10/10 division was made by question: 65,800 responses for training, 8,227 for validation, and 8,237 for testing. By having all responses to the same question stay in the same split, we ensure that a model can't score better simply because it recognized the prompt from training, instead of learning styles. Since models learn from a training set, the validation was used to choose the best checkpoint and stop training, so the test set could only be used once at the end.

Labels and Domains

For RQ3, I grouped questions by the dataset's source benchmark column into three groups: 64,564 responses for everyday conversation, 10,233 for Q&A and instructions, and 7,467 for benchmark prompts. As a reminder, the main label is the six families, with the eight models only used for balancing.

Limitations

There were 3 limitations I had to acknowledge. For instance, for every question that was answered by all AIs, it alone carries no information about who answered it. The three RQ3 groups were all general-purpose AI prompts, thus the difference between them is less obvious compared to that of coding vs poetry for example. Lastly, balancing the models did not mean that the six families were of equal sizes and the dataset only contained specific model versions, so our results may not be applicable to newer models.

II. MODELS AND TRAINING

Preparation

For the responses in the dataset, I lowercased all responses and split them into words and punctuation marks. To preserve formats like lists and paragraphs, I converted line breaks into a special NEWLINE token. I then built a vocabulary of up to 30,000 tokens from the training data alone and for any token appearing fewer than three times, they were mapped to an unknown token. For every response, they were cut or padded to 256 tokens. For RQ2, the question alone was cut or padded to 128 tokens, and the accumulated input consisted of the first 64 question tokens, a separator token, and the first 256 response tokens.

CNN

Each token is converted to a 128-dimension embedding. Afterwards, three convolutional layers with 128 filters and window sizes of 3, 4, and 5 tokens, recognize short patterns, such as characteristic phrases or punctuation. For each filter, max pooling keeps its strongest match across the response, giving 384 values. These values then go through a dropout of 0.5, and a linear layer that outputs a score for each of the six families. The CNN had about 4.0 million trainable parameters, and while it recognizes local patterns well, it may perform weaker with capturing longer-range structures.

LSTM

The LSTM uses the same 128-dimensional embeddings followed by a bidirectional LSTM with 128 hidden units in each direction, so the text can be read forward and backward. For each feature, max pooling keeps its strongest value across the response, followed by dropout of 0.5, and a linear layer for six scores. This has about 4.1 million parameters.

Training

For the training process, both models used a cross-entropy loss and the Adam optimizer, with a learning rate of 0.001 and batch size of 64, for a max of six epochs where each epoch was checked for validation accuracy. If training encountered a failed improvement for two epochs in a row, the run would stop and the checkpoint with the best validation accuracy was recorded and used for testing. I used PyTorch on a MacBook Air with Apple's GPU Support. For the main RQ1 results, I trained each model with three seeds (42, 1, and 2) and reported the mean and standard deviation. All other experiments were done with single runs, and I did no hyperparameter search, so results reflected the settings I chose rather than the full potential of these models. Because GPU training is not perfectly repeatable, reruns of the same setting differed by up to about a point overall and by more on the small Q&A and benchmark test sets.

Baselines

The first baseline used always predicts the largest family with 24.9% accuracy. The second baseline is TF-IDF with logistic regression, using up to 50,000 features from single words and word pairs, including punctuation. This shows how well simple word frequencies can do alone.

III. RESULTS (RQ1 AND RQ2)

RQ1: Can we identify the AI from its response?

For the following table, accuracy is the percentage of test responses assigned to the correct family. Macro-F1 averages the F1 score across the six families to ensure each family counts equally. Results are on the 8,237-response test set.

Model	Accuracy	Macro-F1
Majority-class Baseline	24.9%	-
TF-IDF+ Logistic regression	85.1%	84.2%
CNN (3 seeds)	81.5% ±0.2%	80.0%±0.3%
LSTM (3 seeds)	85.6% ±0.2%	84.8% ± 0.1%

Since both neural models exceed the majority baseline, the families can be identified from their responses. According to the table, the LSTM performed best with 4.1 points above the CNN and 0.5 points above the TF-IDF. Because of this gap, it suggests a simple word frequency model captures most of the same patterns. As for the CNN, it was 3.6 points below TF-IDF possibly due to that it only views short windows of text and possibly benefited from tuning I did not do. Across the seeds, the standard deviations were small, showing that the CNN vs LSTM gap is much larger than the run-to-run variation.

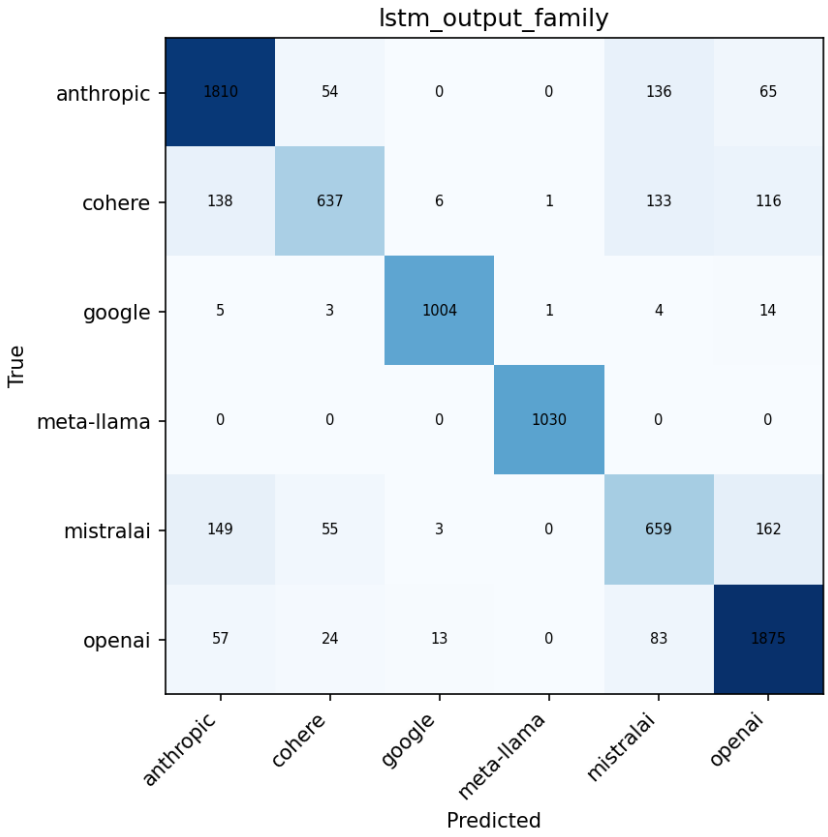


Figure 1: LSTM Confusion matrix on test set (rows= true family, columns=predicted)

The confusion matrix prove that some families are much easier than others, where the LSTM classified these shares of each family correctly: Llama 100%, Google 97%, OpenAI 91%, Anthropic 88%, Mistral 64%, and Cohere 62%. Most of the errors confused Mistral and Cohere with Anthropic, OpenAI or with one another, whereas Llama and Google were

almost never confused. All in all, accuracy varied slightly between runs with the weakest families, Cohere and Mistral, varying the most. Overall, RQ1 shows that AI families can be identified from responses, with the resulting accuracy depending on the model and the level of difficulty of some families compared to others.

RQ2: Does the question help?

For this section, I compared three inputs for each model: question only, response only, and question plus response.

Input	CNN accuracy	CNN Macro-F1	LSTM Accuracy	LSTM Macro-F1
Question Only	25.0%	11.1%	25.0%	11.0%
Response Only	81.5%	80.0%	85.6%	84.8%
Question + Response	83.7%	83.0%	84.9%	84.3%

With question alone, both models provided an accuracy of about 25% with about 11% macroF1, which means it always guessed one family. This is understandable since every question has responses from all eight AIs. Thus, the question carries no information about the source of the response. As for the question plus response, I encountered mixed results with the CNN improving by 2.2 points and the LSTM dropping by 0.7. The CNN's gain is much larger than its seed to seed spread from 81.4% to 81.7%, which means it's probably real, whereas the LSTM's small drop is within the normal variation. However, since the combined runs were only done once, these assumptions should be confirmed with more seeds. A possible reasoning behind the CNN gain is that the question provides context for what style of response is expected, but the response already contains the writing patterns, so the question adds little.

IV. IN DEPTH ANALYSES

RQ3: Do AI fingerprints generalize across domains?

For each domain, I compared two settings. In the seen setting, the model was trained on all three domains and tested on the held-out test rows of the target domain. In the unseen setting, the model was trained on the other two domains and tested on all the target domain's responses.

Target Domain	CNN seen	CNN unseen	LSTM seen	LSTM unseen
Everyday conversation	85.0%	77.3%	87.9%	77.6%
Q&A	69.0%	66.6%	75.5%	72.1%

Benchmark	74.3%	71.9%	78.6%	75.6%

According to the stats, performance dropped in every unseen case, but never by a large percentage: unseen was between 66.6%-77.6% which was far beyond the 24.9% baseline. This means that the accuracy fell somewhat on unseen domains with the small drops for Q&A and benchmark (CNN 2.4 points for both and LSTM 3.4 and 3.0 points respectively),and the largest drops with everyday conversations (CNN 7.7 and LSTM 10.3).

The possible reasoning behind the chat drop could be from a training size effect. While the unseen chat models only trained on about 14,000 responses, with Q&A and benchmark combine, seen models trained on 65,800. To control this, I limited the seen chat models on 14,225 rows, about the same amount of data as the unseen. Due to this, their seen accuracy fell to 80.8% (CNN) and 84.6% (LSTM), so the drops for the domain were 3.5 and 7.0. Despite this, the LSTM still lost more accuracy than the CNN and the two models were nearly tied in their unseen chats (77.6% and 77.3%), which means the LSTM possibly relies more on domain specific patterns. The Q&A and benchmark drops are small and should be read cautiously, since their test sets are small (989 and 775 responses) and reruns of the same setting changed accuracy by 2 to 3 points in some instances.

For some AIs, they were consistently identifiable. For instance, in the unseen chat setting, LSTM was able to identify Llama and Google almost perfectly whereas Cohere was accurate 24% of the time and Mistral responses 32%. Thus, the fingerprint is mixed where it can be strong and general with some families and weaker and domain dependent for others. However, these three domains are general assistant prompts, so these results may differ depending on the task such as coding or writing, and all RQ3 runs were single.

RQ4: What characteristics distinguish AIs?

Length and Formatting

Claude models had the shortest responses of about 191 words and almost never used bold. Llama wrote the longest responses of about 337 words and had the most numbered lists of about 7.5 per response. Gemini had the most bold markers of about 25, meaning roughly 12 bolded phrases, and 10.3 bullets per response. GPT-4o and GPT-4 Turbo had the most headers, about 2.1 and 1.1 per response respectively. This shows that each family has its own length and formatting profile, which allows us to further explain the importance of formatting below.

Opening Phrases

Claude 3 Sonnet started its responses with “Here are some” about 29% of the time. Mixtral started with “Absolutely, I’d be” about 6% of the time, and GPT-4 Turbo started with “As of my” about 3%. Since other models use these words occasionally too, these starting phrases can be labeled as habits rather than unique identifiers.

Simple Features

I built 21 handcrafted features (length, formatting, vocabulary, and punctuation) and trained simple classifiers on them. The table shows the results:

Classifier/ Feature group	**Accuracy**
Gradient boosting, all 21 features	79.7%
Logistic regression, all 21 features	66.9%
Logistic regression, formatting only	55.7%
Logistic regression, length only	39.9%
Logistic regression, vocabulary only	36.3%
Logistic regression, punctuation only	33.5%

For the 21 simple counts, gradient boosting reached 79.7%, close to the neural models, and for the single groups, formatting was the strongest (55.7%) compared to length, vocabulary, and punctuation. When I later dropped one group for logistic regression, removing formatting caused a bigger drop (66.9% to 53.3%) than removing punctuation (64.4%), vocabulary (65.0%), or length (65.8%).

Retraining without signal

Experiment	**CNN**	**LSTM**
Normal	81.5%	85.6%
Formatting removed	68.3%	73.3%
Punctuation removed	74.3%	80.9%
First 50 words only	74.3%	77.0%

After retraining the CNN and LSTM after removing signals from the text, the loss of formatting caused the biggest decline (13.2 points for CNN and 12.3 for LSTM), followed by the 50 word limit (7.2 and 8.6) and punctuation removal (7.2 and 4.7). Every version was above chance, so the signals overlapped where when one is removed, the models can use the others. Much of the signal is concentrated early in the response, since the first 50 words alone kept most of the accuracy.

It must also be noted that “formatting removed” also resulted in the removal of line breaks and symbols, so the measures included a bit more than markdown. And the 50-word test

also removes content along with length, so it's not a pure length test. Also, these were single runs.

Answering RQ4

AI digital fingerprints are a combination of structural and linguistic patterns, with formatting standing out as the biggest signal in all three analyses: feature groups, ablations and retraining. However, one single characteristic is never enough on its own.

V. LESSONS LEARNED

What surprised me

While the question alone did not perform better than guessing, it surprised me that 21 handcrafted counts nearly reached 80%, which is so close to the neural models, and that the TF-IDF baseline almost matched the LSTM. It really showed how much formatting actually mattered when it came to predicting the AI language model, as Llama and Google were almost always predicted correctly while Mistral and Cohere were often confused with other families.

Limitations

Besides the experiments in the main RQ1 runs, everything else used a single seed without any hyperparameter search and a max of six epochs for training. The test sets for Q&A and benchmark were also small, which resulted in a noticeable variation in their reruns. The domains for RQ3 were also all general purpose prompts and the dataset only covered specific model versions, so the results would likely not apply to newer models.

What I would do next

If given the opportunity to do this again, I would tune the models and run multiple seeds for every experiment, especially RQ2. I would also build a cleaner length test that does not remove content and test different domains like coding and poetry.

What I learned from the process

This project showed me that identifying an LLM from its text depends on several overlapping signals, especially formatting, rather than a single one. Splitting the data by question prevented leakage, and using a simple baseline showed that the LSTM's advantage was small.

Use of AI assistance

For this project, I used an LLM, Claude, for coding help, debugging, and further analysis on the concepts I gathered. I ran all the experiments myself and wrote the report based on my own results and used Claude to help revise.